

Strife Engine

Feature Documentation & Game-Building Guide

Companion to `CLAUDE.md` (architecture rules) and `ROADMAP.md` (progress). This file documents what is implemented (M0–M9), how to use each feature, and how to build an actual game on top of the engine.

0 The Big Picture

Strife is a Zig (master/nightly) game engine with:

- **Vulkan 1.3** dynamic-rendering renderer (no render passes), VMA memory, bindless materials, shadows, debug-draw, and a 2D UI overlay.
- A **hand-rolled sparse-set ECS** (not Flecs) — entities, components, queries, systems, events.
- **GLFW** windowing/input, **Jolt** physics, **miniaudio** audio, **cgltf/OBJ** mesh import, **stb** textures/fonts.
- A full **gameplay layer** (health, movement, combat, abilities, inventory, AI, projectiles, save/load).

Everything runs through one loop. `main.zig` builds a `Config`, creates `Engine(VulkanWorld)`, and calls `run()`. The `VulkanWorld` owns the Vulkan context, the ECS `Registry`, and a `SystemManager` that ticks every registered system in priority order at a fixed `1/60 s` timestep.

```
main.zig → Engine(VulkanWorld).init/run/deinit
    └─ VulkanWorld: VulkanContext + Registry + SystemManager(all_systems)
        └─ each fixed step: reset scratch arena → SystemManager.update(dt)
```

The single source of truth for "what runs and in what order" is `all_systems.zig`. Systems update in **ascending priority**; they are created in **descending priority** and destroyed in reverse create order.

1 Foundation Layer (M0)

Located in `src/engine/`. These are the primitives every other layer builds on.

Feature	File	How to use
Window	<code>platform/window.zig</code>	GLFW wrapper. <code>Window.framebufferSize</code> , <code>createSurface</code> , <code>setCursorMode</code> , <code>getKey</code> / <code>getMouseButton</code> / <code>getCursorPos</code> , <code>pollEvents()</code> . Engine code goes through this, never raw <code>glfwXxx</code> .
Main loop	<code>engine/engine.zig</code>	Fixed timestep accumulator, <code>FIXED_DT = 1/60</code> , capped at <code>MAX_STEPS_PER_FRAME = 5</code> . You don't call this — you register systems.
Input	<code>engine/input.zig</code>	<code>InputState.isDown/justPressed/justReleased</code> , edge-detected each frame. GLFW-decoupled (takes an <code>anytype</code> key source).
Timer	<code>engine/timer.zig</code>	<code>Timer.tick()</code> / <code>elapsed()</code> -> <code>f64</code> . Built on <code>std.io.Clock</code> .
Logging	<code>engine/log.zig</code>	<code>Level</code> enum, <code>@src()</code> file:line, ANSI color. Debug-level calls compile out in release.
Assertions	<code>engine/assert.zig</code>	<code>strife_assert(cond, msg, @src())</code> . Compiles out in non-Debug.
Filesystem	<code>engine/fs.zig</code>	<code>readFileAlloc</code> , <code>writeFile</code> , <code>fileExists</code> , <code>makeDirs</code> , <code>pathExt</code> , <code>pathStem</code> , <code>pathJoin</code> . Built on <code>std.io.Dir</code> .
Math	<code>engine/math.zig</code>	<code>Vec2/3/4</code> (<code>@Vector</code>), <code>Mat4</code> (column-major), <code>perspective</code> , <code>lookAt</code> , <code>transformToMatrix</code> . Mat4 is column-major; perspective negates Y and uses depth [0,1].
Memory	<code>engine/pool.zig</code> + world arena	<code>PoolAllocator(T)</code> (index-handle free list). Per-frame scratch is <code>VulkanWorld.scratch_arena</code> , reset every step, reachable via <code>SystemCreateCtx.scratch</code> .
Jobs	<code>engine/jobs.zig</code>	<code>JobSystem.submit()/waitAll()</code> , built on <code>std.io.Group</code> . Use for batch work where blocking until done is acceptable.
Config/INI	<code>engine/ini.zig</code> + <code>engine/config.zig</code>	<code>loadFromIni()</code> overlays <code>strife.ini</code> onto <code>Config.default</code> . See §2.
UUID	<code>engine/uuid.zig</code>	<code>Uuid.v4()</code> , <code>toBuf()</code> -> <code>[36]u8</code> , <code>parse()</code> .

⚠ Zig master gotcha (also in `CLAUDE.md`): `std.time.Timer`, `std.fs.cwd()`, `std.Thread.sleep/Mutex`, `std.crypto.random` are all gone — replaced by `std.io.*` equivalents that take an `Io` instance (`std.io.Threaded.init(allocator, .{}).io()`). Always fetch master docs before writing Zig.

2 Configuration — `strife.ini` + `Config`

`Config` is the engine's startup knob set. `main.zig` overlays `strife.ini` (repo root) onto `Config.default` before constructing the engine.

Key fields:

```
Config{
    .window_title, .window_width, .window_height,
    .max_frames_in_flight = 2, .max_textures = 1024,
    .enable_validation = true, .vsync = true,
    .hot_reload_shaders = (Debug only),
    .camera = CameraConfig{ .position, .target, .near, .far },
    .lighting = LightingConfig{ .direction, .color, .ambient, .shadow_half_extent, ... },
    .audio = AudioConfig{ .master_volume, .ui_volume, .sfx_volume, .music_volume },
    .scenes = []const SceneConfig{ ... }, // static glTF/OBJ scenes preloaded at startup
}
```

A `SceneConfig` declares a static scene to preload (path, camera, world offset, whether it auto-rotates, and an optional spatialized audio clip). Currently wired INI sections: `[window]` (width/height/vsync), `[engine]` (validation), `[audio]` (the four volumes). Extend `ini.loadFromIni` to thread more fields through.

To add a static scene to your game: add a `SceneConfig` entry to `Config.default.scenes`. The Scene system preloads and GPU-uploads every configured scene once at startup and keeps them all resident (so switching is instant).

3 The ECS — Entities, Components, Queries, Systems, Events

This is the heart of how you write game logic. Files in `src/engine/ecs/`.

3.1 Registry API

```
const Registry = @import("engine/ecs/entity/registry.zig").Registry;
const components = @import("engine/ecs/components/components.zig");

const e = try registry.create(); // new entity {index, generation}
try registry.add(e, components.TransformComponent{ ... });
try registry.set(e, components.TransformComponent{ ... }); // overwrite (deinit old if owned)
const t = registry.get(components.TransformComponent, e); // ?*T — mutate in place
registry.remove(components.MeshComponent, e);
try registry.destroyEntity(e); // frees owned data, emits .entity_destroyed
const alive = registry.isAlive(e);

// Query iterates the SMALLEST matching storage, mask-filters the rest. No per-frame alloc.
var it = registry.Query(.{ components.MeshComponent, components.TransformComponent });
while (it.next()) |entity| {
    const tr = registry.get(components.TransformComponent, entity).?;
    // ...
}
```

Entities are generational handles; destroyed indices recycle with bumped generations, so stale handles never alias. Component lookups via `get` return a mutable pointer you write through directly.

3.2 Adding a new component

1. Add the POD struct to `components.zig`.
2. Add the type to the `AllComponents` tuple at the top.

That's it — `ComponentBit` / `ComponentIndex` and storage are derived at compiletime. **Components must be POD** (no allocator fields, no heap pointers). If a component owns memory (rare — e.g. `PoseBufferComponent`), give it a `deinit(self, allocator)` method and the registry calls it on `set` / `remove` / `destroyEntity` / `deinit`.

3.3 Writing a system

A system is three free functions + a `SystemDesc` entry in `all_systems.zig`. The minimal shape:

```

pub fn create(ctx: *SystemCreateCtx) anyerror!*anyopaque {
    // allocate per-system state, subscribe to events, grab ctx.config/registry/scratch
    const state = try ctx.allocator.create(MyState);
    state.* = .{ ... };
    return @ptrCast(state);
}

pub fn update(registry: *Registry, ctx: *anyopaque, dt: f32) anyerror!void {
    const state: *MyState = @ptrCast(@alignCast(ctx));
    var it = registry.Query(.{ ... });
    while (it.next()) |e| { ... }
}

pub fn destroy(allocator: std.mem.Allocator, registry: *Registry, ctx: *anyopaque) void {
    const state: *MyState = @ptrCast(@alignCast(ctx));
    allocator.destroy(state);
}

```

Then register it in `all_systems.zig`:

```

.{ .name = "MySystem", .priority = 17, .create_fn = my.create, .update_fn = my.update, .destroy_fn = my.destroy },

```

Priority is everything. Systems update in ascending priority order; they are created in descending priority and destroyed in reverse. This defines the dependency graph of your game logic.

3.4 Events

A typed in-process `EventBus` on `registry.events`. Subscribe in `create`, emit anywhere:

```

try registry.events.subscribe(.damage_event, ctx_ptr, callbackFn);
registry.events.emit(.{ .damage_event = .{ .target = e, .amount = 30, .source = attacker } });

```

Event types: `entity_destroyed`, `scene_unloaded`, `anim_event`, `trigger_event`, `damage_event`, `death_event`, `footstep_event`, `hit_reaction_event`. This is the substitution for Flecs observers throughout the codebase — "emit → subscriber", not "emit → observer".

4 Rendering (M3)

You generally **don't call the renderer directly**; you attach components and the Render system (priority 100) draws them. Give an entity a `MeshComponent{ mesh_id }`, a `MaterialComponent{ material_index }`, and a `TransformComponent`, and it renders.

- **Vulkan 1.3 dynamic rendering**, double-buffered (`FRAMES_IN_FLIGHT = 2`), sync2 barriers, VMA memory.
- **Bindless materials**: a single `StructuredBuffer<MaterialData>` indexed by a push-constant `materialIndex`; a parallel bindless texture array. One pipeline today.
- **Camera**: `CameraComponent` + `CameraMatricesComponent`, persistently-mapped UBO. Blinn-Phong lighting from `Config.lighting`.
- **Shadows**: 2048² D32 shadow map, orthographic light-space matrix, 3×3 PCF. (*Known gap: the shadow pass does not skin — animated meshes cast a bind-pose silhouette.*)
- **Debug draw**: any system with priority < 100 can call `ddLine` / `ddAxes` / `ddBox` / `ddSphere`. Accumulated per step, drained per frame. **Use this to visualize new systems before you have UI.**
- **Shader hot reload** (Debug): edit a `.slang`, recompile, and the running app hot-swaps pipelines.

Shaders are **Slang**, not GLSL, in `src/shaders/`.

5 Assets (M2)

Asset	Loader	Notes
Mesh (glTF)	<code>resources/meshLoader.zig</code> (cgltf)	→ <code>GpuMesh{vertexBuffer, indexBuffer, indexCount}</code> . Reads <code>JOINTS_0</code> / <code>WEIGHTS_0</code> for skinning.
Mesh (OBJ)	<code>resources/objLoader.zig</code>	v/vn/vt/f, wrapped as a one-mesh <code>GltfScene</code> so it routes through the same pipeline. Sibling <code><name>.json</code> supplies the material.
Texture	<code>material.zig</code> (stb_image)	<code>R8G8B8A8_SRGB</code> , full mip chain via <code>vkCmdBlitImage</code> .
Material	<code>resources/materialLoader.zig</code>	JSON <code>{ albedo, metallic, roughness }</code> → bindless <code>MaterialData</code> .
Skeleton/anim	<code>animation/gltf_import.zig</code>	<code>loadSkin</code> / <code>loadAnimationClip</code> → joint-sorted <code>Skeleton</code> + <code>AnimationClip</code> .
Generic async	<code>engine/assets.zig</code>	<code>AssetManager(T)</code> with <code>Handle</code> , refcount, async load. Not yet the path the concrete mesh/texture pipeline uses.

Animation (M4)

- **Skeletons:** joints topologically sorted at load, so skinning is one forward sweep. `bindPose()` returns rest-pose locals.
- **Clips:** `AnimationClip{ name, duration, channels, events }`, decomposed TRS keyframes, `sampleClip(clip, time, out_poses)` with lerp/slerp. `blendPoses(a, b, alpha, out)` for crossfades.
- **Blend tree:** `BlendTree1D` — bracket-and-blend on a 1D param (e.g. speed → idle/walk/run). Logic only, no ECS component yet.
- **State machine:** `StateMachine` with conditioned transitions and crossfade. Logic only, no ECS component yet.
- **Anim events:** `forEachFiredEvent` emits `.anim_event` through the EventBus as playback crosses an event's time.

ECS wiring: attach `SkeletonComponent{skeleton_id}` + `AnimPlayerComponent{clip_id, time, speed, loop}` + `PoseBufferComponent` + `SkinPaletteComponent`; the Scene system attaches all four automatically for any spawned primitive whose asset has a skin+clip. GPU skinning blends up to 4 joints/vertex in `shader.slang`.

Gap to close for real character animation: blend-tree/state-machine are not yet ECS components, and the shadow pass doesn't skin. Locomotion blending waits until gameplay needs it.

7 Physics (M5)

Jolt via the flat `extern "C"` wrapper (`src/physics/jolt_wrapper.cpp` / `.h`); Zig never touches a Jolt C++ type. Bindings are **pre-generated** in `jolt_wrapper.zig`.

```
const PhysicsWorld = @import("physics/physics_world.zig").PhysicsWorld;
// One PhysicsWorld per World, published via physics_shared.zig (module-global *PhysicsWorld).

world.spawnBoxBody(...);           // dynamic/static box → body_id
world.despawnBody(body_id);         // MUST call before destroyEntity
world.overlapSphere(center, radius, out_entities) usize; // melee/AOE queries
world.applyImpulse(body_id, impulse); // knockback

const hit = raycast(origin, dir, max); // physics/raycast.zig → ?RaycastHit
raycastAll(...);                      // multi-hit (capped 64)
```

- **PhysicsBodyComponent** + PhysicsSync (20) steps Jolt once/tick and writes non-static bodies back to `TransformComponent`.
- **CharacterControllerComponent** (Jolt `CharacterVirtual`, capsule) + CharacterController (21). The caller integrates gravity, `jump()` only fires when grounded. `setVelocity` / `isGrounded`.
- **Triggers:** sensor bodies via `spawnBoxTrigger`; `TriggerWatcherComponent` marks them; Trigger system (22) emits `.trigger_event{ trigger_ent, other_ent, is_enter }`.
- **Collision layers:** `static/player/enemy/projectile/trigger`, matrix baked into Jolt at init.

⚠ **Memory rule:** `PhysicsBodyComponent` and `CharacterControllerComponent` can't auto-free through the registry's generic deinit hook. **Always call** `despawnBody` / `despawnCharacter` **before** `destroyEntity`, or you leak the Jolt side.

Prefabs

`PrefabRegistry` is a module-global. It auto-loads every `assets/prefabs/*.json` (each `{name, mesh_path}`) at startup. Instantiate at runtime:

```
const id = prefab.global?.idByName("goblin").?;  
const root = try prefab.global?.instantiate(registry, id, transform);  
// multi-primitive assets get a root + ParentComponent-linked children  
prefab.global?.destroyInstance(registry, root);
```

GPU assets are cached by `mesh_path`, so repeat instantiation is free.

Spawners

Attach `SpawnPointComponent{prefab_id, cooldown, max_active}`. The Spawner system (1) ticks the timer and instantiates up to `max_active` live instances. `active_count` self-corrects via an `entity_destroyed` subscription.

Scene save/load

Persists spawn points + prefab instances (by **name**, not id; UUIDs round-trip) to JSON. Static geometry is **not** re-serialized — only the `scene_path` is recorded and reconstructed by re-running the preload pipeline.

Audio (M7)

miniaudio via the `miniaudioimport` translate-c module; Zig touches `ma_*` only inside `audio_device.zig`.

- **AudioEngine/AudioClip:** `clipLoad` / `clipPlay` / `clipIsPlaying`. **Init in place** — these structs hold self-pointers and must never be returned/copied by value.
- **AudioClipCache:** dedups by path, stores `*AudioClip` (heap-allocated).
- **AudioSourceComponent:** `{clip_id, volume, auto_play, playing, spatialized, rolloff, min_distance, max_distance}`. Audio system (4) plays `auto_play` once. Audio3D system (61) tracks the camera as listener and spatializes sources.
- **Mixer:** three buses (`ui/sfx/music`) as `ma_sound_group`s + engine master. `setVolume(bus, v)`. A clip's bus is fixed at load time.

Caveat: two sources sharing a `clip_id` share one `ma_sound` (and its position/spatialization state). Fine for one-shot SFX; a real sound pool is needed for the same clip playing spatially from two places at once.

UI (M8)

A 2D overlay drawn last each frame through a dedicated `ui.slang` pipeline (screen-space ortho, alpha blended, no depth). All share one bindless texture array with the main pass.

- **Font:** `Font.load` bakes a TTF (stb_truetype) into a 1024² RGBA atlas. `GlyphInfo` per glyph.
- **Text:** `drawText` / `measureText`.
- **Images:** `drawRect` / `drawImage` / `drawImageUV`. Solid rects reuse the engine's default 1×1 white texture.
- **Buttons:** `ButtonWidget{pos, size, label, state, on_click}`, `containsPoint`, normal/hover/pressed driven by direct cursor/mouse polling; fires `on_click` on release-while-hovering.
- **Health bars:** `worldToScreen` projects a world point through a `view_proj`; `draw()` shows a red→green fill, hides at full health.

UI is currently **immediate-style and driven from harness/world code** — there's no retained UI-tree ECS component yet. For game menus/HUD you call these draw helpers from a system or from `world.zig`.

11 Gameplay Layer (M9)

All in `src/gameplay/`. This is the toolkit you assemble a game from. Each is a real ECS system already registered in `all_systems.zig`.

System	Component(s)	How to drive it
Health	<code>HealthComponent{current, max, regen_per_sec, invincible, invincible_timer, iframe_duration}</code>	Emit <code>.damage_event</code> . System applies damage, fires <code>.death_event</code> once, <code>.hit_reaction_event</code> on every landing hit, grants i-frames.
Movement	<code>PlayerMovementComponent</code> + <code>CharacterControllerComponent</code>	Populate <code>shared_state.PlayerInput{move_forward, move_right, sprint, jump_pressed}</code> (Input system does this from WASD/space/shift). Camera-relative; emits <code>.footstep_event</code> .
Combat	<code>MeleeAttackComponent{range, radius, damage, impulse, cooldown, trigger}</code>	Set <code>trigger = true</code> . System does a sphere overlap from attacker facing, emits <code>.damage_event</code> + impulse. Generic to player or enemy.
Abilities	<code>AbilitySlotsComponent{slots[6], resource, request_cast, request_target}</code>	Define <code>assets/abilities/*.json</code> (<code>{name, cooldown, resource_cost, cast_time, effects:[{kind, amount}]}</code>). Set <code>request_cast = slot</code> . Effects: <code>damage</code> / <code>heal</code> / <code>knockback</code> .
Inventory	<code>InventoryComponent{items[20], relics[3], request_use_slot}</code> , <code>PickupComponent</code> , <code>ItemStack</code>	Define <code>assets/items/*.json</code> . Pickups are trigger volumes; walking an inventory entity into one grants the item. Set <code>request_use_slot</code> to consume.
AI	<code>AIComponent{state, patrol_points, sight_range, attack_range, retreat_health_fraction, ...}</code>	Flat per-state switch: patrol→chase→attack→retreat→dead. Targets the nearest <code>PlayerMovementComponent</code> entity with line of sight. Reuses <code>MeleeAttackComponent</code> .
Projectiles	<code>ProjectileComponent{velocity, damage, owner, lifetime, impulse, impact_prefab_id}</code>	Spawn an entity with this; system sweeps a raycast each frame, excludes <code>owner</code> 's first hit, emits damage + impulse + optional impact-VFX prefab.
Save/Load	<code>gameplay/save_system.zig</code> + <code>FlagSet</code> (<code>save_system.global</code>)	F5 quicksave / F9 quickload via <code>shared_state.SaveRequest</code> ; autosave timer (60 s). Saves the player's health/transform/inventory/relics (items by name) + named progression flags to <code>saves/slot_N.json</code> .

Module-global registries you'll use from game code: `prefab.global`, `ability.global`, `item.global`, `save_system.global`.

Helper functions: `ability.applyEffects(registry, effects, caster, target)`, `item.addItem(ireg, inv, item_id, count) bool`.

12 How to Build a Game on Strife

The engine is at the **M10 line**: "stop engine work, build Strife gameplay only." Here's the practical path.

12.1 Where game code lives

Game-specific logic is just **more ECS systems** in `src/gameplay/` (or a new `src/strife/` folder), registered in `all_systems.zig`. Don't reach into the renderer/platform from gameplay; drive everything through components + events + the module-global registries.

12.2 A minimal vertical slice — step by step

1. Author assets as data.

- Static level: add a `SceneConfig` to `Config.scenes` (glTF/OBJ path + camera).
- Enemies/props: `assets/prefabs/*.json` (`{name, mesh_path}`).
- Abilities: `assets/abilities/*.json` . Items: `assets/items/*.json` .

2. **Spawn the player.** Write a small "GameSetup" system (low priority, e.g. -50) whose `create()` spawns one entity with: `TransformComponent` , `CameraComponent` (or a follow-cam you write), a Jolt `CharacterControllerComponent` , `PlayerMovementComponent` , `HealthComponent` , `InventoryComponent` , `AbilitySlotsComponent` , and a `MeleeAttackComponent` . The Input system already fills `PlayerInput` ; `PlayerMovement` will drive the character.

3. **Populate the world.** In the same setup system, instantiate prefabs and give enemies `AIComponent` + `CharacterControllerComponent` + `HealthComponent` + `MeleeAttackComponent` . Drop `SpawnPointComponent` s for waves. Place `PickupComponent` trigger volumes for loot.

4. **Wire game rules with events.** Subscribe to `.death_event` (score, drop loot, respawn), `.damage_event` (screen shake), `.footstep_event` (play a sound), `.trigger_event` (doors, checkpoints). This is where your game's *feel* lives.

5. **HUD.** From a system running before Render, call `text_renderer.drawText` , `image_renderer.drawRect` , and `health_bar.draw` each frame. Use `ButtonWidget` for menus.

6. **Persistence.** Set named flags through `save_system.global` for quest/progression state. Quicksave/load already works.

7. **Iterate visually with debug draw.** Before art exists, `ddBox` / `ddSphere` / `ddAxes` every new system so you can see it working.

12.3 Known gaps (good first game-layer tasks)

- Animation blending in ECS:** wrap `blend_tree.zig` / `state_machine.zig` in a component so characters blend idle/walk/run/attack.
- Skinned shadows:** the shadow pass doesn't read the skin buffer — animated shadows show the bind pose.
- A dedicated PlayerTag:** today "the player" is "the entity with `PlayerMovementComponent` ." A tag is cleaner once there are multiple player-like entities.
- Factions/threat tables:** AI currently targets any `PlayerMovementComponent` entity. Add a faction component when you have allied NPCs.
- Sound pool:** same clip from two spatial locations at once needs instance separation in `AudioClipCache` .
- Physics in prefabs/save:** prefab JSON has no collision-shape field yet, so save/load can't recreate bodies.
- Targeting system:** abilities/projectiles default to self/forward; a real target-acquisition system would make ranged combat richer.

12.4 Build & run

```
zig build                # debug
zig build -Doptimize=ReleaseFast
zig build run
cd src/shaders && ./compile.bat # only if you edit a .slang by hand
zig build test           # full test suite
```

13 Quick Reference

"I want to..."

Goal	Do this
Render a model	Entity + MeshComponent + MaterialComponent + TransformComponent
Add a static level	SceneConfig in Config.scenes
Make a reusable spawnable	assets/prefabs/x.json → prefab.global?.instantiate
Periodic enemy waves	SpawnPointComponent
A moving player	PlayerMovementComponent + CharacterControllerComponent ; Input fills PlayerInput
Damage something	registry.events.emit({ .damage_event = ... })
Melee attack	set MeleeAttackComponent.trigger = true
Cast an ability	assets/abilities/x.json + set AbilitySlotsComponent.request_cast
Give/use an item	assets/items/x.json + PickupComponent / request_use_slot
An enemy that fights	AIComponent + CharacterControllerComponent + MeleeAttackComponent + HealthComponent
Fire a projectile	spawn entity with ProjectileComponent
Play a sound	AudioSourceComponent (auto_play) or audio_device.clipPlay
3D positional sound	AudioSourceComponent{ .spatialized = true } + a transform
Draw HUD text/health	text_renderer.drawText , health_bar.draw
Save progress	F5/F9, or set flags via save_system.global
Visualize a new system	ddBox / ddSphere / ddAxes via render_system wrappers
Run physics queries	raycast , world.overlapSphere
React to game events	registry.events.subscribe(...) in a system's create

Generated from the M0–M9 implementation. For architecture rules (column-major Mat4, sync2 barriers, POD components, GLFW/Jolt/miniaudio boundaries) see `CLAUDE.md`. After M9, engine work stops — everything above is the surface you build the game against.

